Computer Science and Engineering Department, SVNIT, Surat
M.Tech. I – Semester I
Design and Analysis of Algorithm (CSDS103)
Lab Assignment 7

General Instructions

1. Implement the specified string matching algorithms without using built-in pattern searching functions.
2. Analyse the time and auxiliary space complexity of each algorithm.
3. Test the algorithms using different text and pattern sizes and different input characteristics.
4. Record execution times and generate suitable performance graphs to compare experimental and theoretical behaviour.

Problem 1 – Naive String Matching

Given a text string T of length n and a pattern P of length m, find all occurrences of P in T using the Naive String Matching algorithm.

For example, for

$$T = \texttt{AABAACAADAABAABA}$$

and

$$P = \texttt{AABA},$$

report all positions at which the pattern occurs.

1. Implement the Naive String Matching algorithm.
2. Count the number of character comparisons performed during the search.
3. Test the algorithm using different text and pattern lengths.
4. Analyse the best-case and worst-case time complexity and auxiliary space complexity.
5. Identify input patterns that result in a large number of character comparisons.

Analysis:

Construct a text and pattern for which the algorithm performs close to its worst-case number of comparisons. Explain why the repeated comparisons occur.

Problem 2 – Knuth–Morris–Pratt (KMP) String Matching

Implement the **Knuth–Morris–Pratt (KMP)** algorithm to find all occurrences of a pattern P in a text T.

Before performing the search, construct the **LPS (Longest Proper Prefix which is also Suffix)** array for the pattern.

For example, for the pattern

```
ABABCABAB
```

determine its LPS array and use it during pattern matching.

1. Construct the LPS array without using any built-in string matching function.
2. Use the LPS array to find all occurrences of the pattern.
3. Count the number of character comparisons performed by the algorithm.
4. Analyse the time and auxiliary space complexity of KMP.
5. Compare the number of comparisons performed by KMP with the Naive String Matching algorithm for the same inputs.

Problem 3 – Rabin–Karp String Matching

Implement the **Rabin–Karp** algorithm to find all occurrences of a pattern P in a text T using a rolling hash function.

The algorithm should compare the hash value of the pattern with the hash value of each substring of the text having the same length as the pattern. Character-by-character verification must be performed when the hash values match.

1. Design an appropriate hash function for the strings.
2. Implement the rolling hash so that the hash value of the next substring can be obtained efficiently.
3. Count the number of hash matches and character-by-character verifications.
4. Analyse the average-case and worst-case time complexity and auxiliary space complexity.
5. Compare the execution time of Rabin–Karp with Naive String Matching and KMP for different text and pattern sizes.

Analysis:

Construct or identify an input that produces hash collisions. Explain why a hash match does not necessarily imply that the pattern actually occurs at that position. Discuss how the choice of the hash parameters affects the performance of Rabin–Karp.

Problem 4 – Experimental Comparison of String Matching Algorithms

Generate suitable graphs such as:

- Input Size vs Execution Time
- Pattern Length vs Execution Time
- Input Size vs Number of Comparisons